

The Cost of Security: Quantifying the Security-Performance Tradeoff in Linux DPDK Environments

Nikhil Jayakumar

December 5, 2025

Abstract

Describe the overall area of contribution, the crux of the problem, and end with highlights of results.

For the initial report, end with the proposed experiments and what you aim to find out.

1 Introduction

The demand for high-throughput and low-latency networking is everlasting, especially with the rise of cloud computing and edge network services. Specialized hardware allows for high-bandwidth data transfers as well as Remote Direct Memory Access (RDMA) to accelerate heterogeneous workloads and network-driven applications. However, for these speeds to be realized, corresponding software needs to be processed at the same speed. To meet this requirement, traditional operating system networking stacks, which were designed around security and control, are often bypassed. An example of such technologies is the Linux Data Plane Development Kit (DPDK) which bypasses the kernel to eliminate the cost of user-kernel context switching, memory copies, and general purpose packet processing. This bypassing has allowed anywhere from 3-10x [VT23] speedup.

This endless demand for performance has often come at the cost of security. Kernel-level processing allowed for validations such as input/output virtual address translation, TCP/UDP checksum verification, and anti-spoofing checks. When bypassing the kernel, the user application is left vulnerable to malicious or erroneous data. Prior research has extensively quantified the performance gain of various kernel bypass technologies, but often fail to account for the lack of security sacrificed with such performance gains. By re-implementing these kernel security measures back into DPDK, this work shows whether the benefit of DPDK is lost when applications need to be secure and reliable.

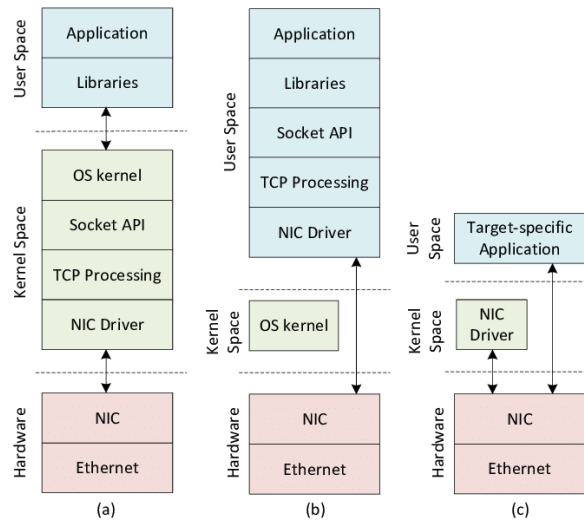


Figure 1: Example Kernel Bypass Architecture [GKCS22]

In order to correctly quantify the performance penalty seen by security protocols, we re-implement these same security protocols and measure the performance from various angles such as throughput, CPU cycles per packet, and latency. Furthermore, we quantify these penalties with granular levels of verification (e.g. verify just the TCP checksum). This granular approach provides a novel understanding of the security-performance tradeoff in Linux DPDK. Aside from specific protocols, we look into various methods of these security implementations, such as VirtIO offloading or completely virtual based checks. This systematic and granular measurement of security protocols in the Linux DPDK presents a novel perspective on the true benefit of Linux DPDK for secure and performant applications.

To formally quantify this trade-off, we split our experiments into three research questions. First, we measure the throughput (packets per second) and CPU cycles per packet of various levels of secure packet processing. Specifically, we measure 4 different levels including Layer 3 verification and Layer 4 Verification (discussed in section 3). Second, we measure the difference of Guest vs. Host Security protocols. We utilize the same levels of secure packet processing, but apply to both the host bare metal and the guest virtual machine. Finally, we measure the resilience of DPDK against malicious and erroneous packets. Not only can this corrupt and destroy user applications, but it could destroy DPDK performance gains due to cache misses or incorrect memory address translations.

The findings from this evaluation provide developers with tremendous insights into the benefits of DPDK and kernel bypass techniques for secure and reliable applications. For instance, developers can understand the specific performance-security tradeoff they are willing to tolerate for their application, and design their application with specific guidelines about guest vs. host based security validation. Future works can include a more efficient secure kernel bypass or more security protocols added onto Linux DPDK. The remainder of this report details the motivation, the experimental setup and design, results and evaluation, related work, and conclusion.

2 Background and Motivation

The motivation of this project is twofold: first quantifying the security-performance tradeoff as well as providing simple security checks for DPDK applications. This section details the background and motivation of kernel-bypass technologies as well as their need for enhanced security.

The Network Interface Card (NIC) is the first place incoming frames hit on a machine. The NIC matches the Layer 2 MAC address and verifies the checksum before making a direct memory access to move the frame into kernel memory. The NIC also has its own small section of memory where it stores circular queues for transmitting and receiving frames, known as TX and RX ring buffers. These buffers simply store DMA descriptors and addresses of packets in memory. Then the NIC triggers a kernel interrupt [Riz12].

This kernel interrupt is split into two halves. The top half is simply the interrupt service routine, designed to be quick and minimal. Therefore, the main parts of this application is to simply acknowledge the interrupt and schedule the bottom half to the CPU. The bottom half is what actually processing and verifies the packet. This includes checksum verification, layer 4 (TCP/UDP) processing, layer 3 (IP) processing, and finally sending to the application socket [GEW⁺15].

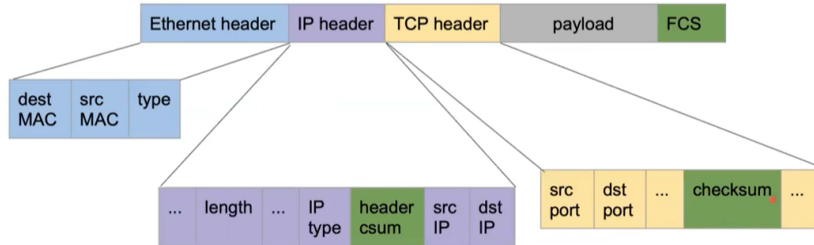


Figure 2: Network Packet Contents [SN16]

A complex and expensive operation done by the kernel is the checksum validation. For instance, the TCP checksum requires a re-construction of a "pseudo-header" which uses various layers of the packet header (IP addresses, message length). Along with the algorithmic complexity, this process

can result in expensive memory access patterns. Although some NICs can offload this computation, it is often more secure to have it calculate it within the kernel. Other solutions involve offloading the checksum validation offloading on custom hardware (i.e. FPGA) [SRLBA18].

The kernel network stack is expensive due to its interrupt-driven nature and packet processing cost. Packet processing is often split into two halves, a top half (interrupt service routine) and a bottom half (packet processing). With higher packet rates, the top half can be too expensive on the CPU such that there is little room left by the bottom half (known as "Receive Livelock") [Riz12]. Polling-mode device drivers, used by kernel-bypass technologies including DPDK, are much faster due to less kernel-user context switching overhead [ASVT18]. Kernel bypass technologies further improve by bypassing most of the "bottom-half" of the packet processing. It then forces the end application to handle any packet processing. Other kernel-bypass technologies such as Vector Packet Processing (VPP) extend DPDK by using batches (vectors) to amortize the cost of fetching network instructions for the CPU [BLM⁺18].

The obvious gap seen by these kernel-bypass technologies is the lack of security or validation done by packets. This forces the user application to perform these security checks, which can have adverse effects on not only the application but even the underlying OS/machine. Other works acknowledge the lack of isolation and security seen by kernel-bypass technologies [SAPK23], but many assume the end-application will handle these steps [Z⁺18, HZLM16, SLW⁺20]. However, [OLL20] enhances IPSec with DPDK, which is a different approach to a similar problem that this work solves. This work provides more granular and fine-grained security, and allows the user to directly modify the security-performance tradeoff of their application.

This work bridges the gap between security and performance with a thorough analysis of integrating security in kernel-bypass technologies. Furthermore, this work will provide simple security checks to allow DPDK developers to integrate into their applications.

3 Proposed Design or Architecture

All the experiments are conducted on a single physical host that emulates a cloud NFV environment. This allows for fine-grained control over the hardware and software along with accurate measurements of the CPU and virtual network devices.

The experiments for the first research question were run on a bare metal device with virtual network interface cards. Figure 1 shows a component diagram of the experimental setup showing both the physical hardware and the software running on it. Specifically, Ubuntu 24.04 was installed on a Ryzen 5 5600u CPU. DPDK was built from source and a modified version of the Layer 2 Packet Forwarding (L2FWD) example was used. This example creates two ports and sends packets back and forth between ports. The example was modified to generate packets if none were received, allowing this test to be a standalone baseline to measure packet processing speed. All further experiments were based off of this modified L2FWD test.

Our evaluation metrics mainly consist of performance, hardware usage, and security. Therefore, all experiments will be evaluated with 4 main metrics: latency, throughput, CPU utilization, and a holistic threat model to measure security.

The first research question adds a granular verification to the L2FWD test, and allows the user to pick what level of granularity they prefer. Figure 1 describes the different levels of verification we will be testing. These levels will be measured and evaluated through the metrics described above. It can be assumed that increasing the level of verification will simply degrade the performance of DPDK. However, we hypothesize that DPDK will still be beneficial to the end user despite the security overhead.

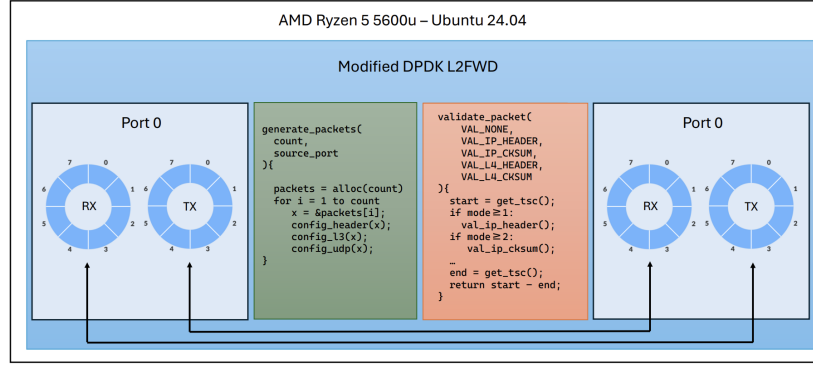


Figure 3: Modified DPDK L2FWD for Granular Packet Validation.

The second research question applies the same experimental setup but added a virtualization layer to measure the difference between Guest side validation and VirtIO offloads. This is done through a QEMU VM running on the machine, and L2FWD is ran on the VM. The host machine does not run any DPDK applications, but performs validation offloaded by the VM through VirtIO. We hypothesize that this simple change can drastically change our metrics due to general context switching and Host-VM communication overhead.

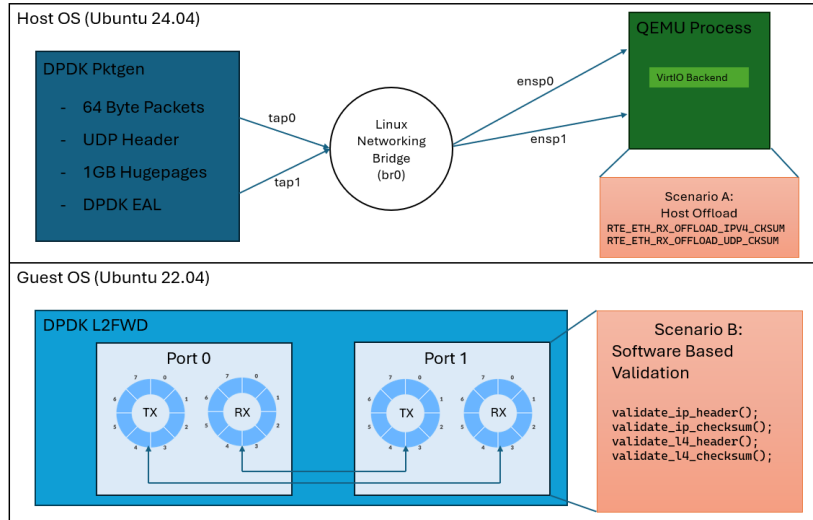


Figure 4: Guest vs. Host Validation Architecture.

The final research question will focus on applying malicious and/or erroneous packets to the first research question. Instead of generating valid checksums and packet headers, this test will measure how much malformed packets affect the performance of DPDK. We are concerned and hypothesized that malformed kernel packets affect cache lines, IOTLB lookups, and other architecture-level penalties. Furthermore, this can be used along with the the first research question to further measure how a variety of packets (malformed or not) affect DPDK performance with different levels of packet verification.

4 Evaluation / Experimental Results

A series of subsections – use the WALTER guide as a simple baseline. Why/purpose of the experiment; explain the Axes, the Lines; the Trends; the Exceptions; and the Result/conclusions you can draw from this experiment.

5 Related Work

This is for related work in depth, so it does not break up the direct storyline up front.

6 Conclusions

What conclusions can you draw from your project – this is not a summary of the key results. Extrapolate into the future. Examine if your results lead to a re-evaluation of current tropes. The conclusions have to be interesting – ideally – instead of being a summary of some facts.

References

- [ASVT18] A. Al-Sbo, M. Vorbrodt, and I. Toivanen. A High-Performance Implementation of an IoT System Using DPDK. *MDPI*, 8(4):550, 2018.
- [BLM⁺18] David Barach, Leonardo Linguaglossa, Damjan Marion, Pierre Pfister, Salvatore Pontarelli, and Dario Rossi. High-Speed Software Data Plane via Vectorized Packet Processing. *IEEE Communications Magazine*, 56(8):97–103, 2018.
- [GEW⁺15] Sebastian Gallenmüller, Paul Emmerich, Florian Wohlfart, Daniel Raumer, and Georg Carle. Comparison of Frameworks for High-Performance Packet IO. *2015 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pages 29–38, 2015.
- [GKCS22] Jose Gomez, Elie Kfoury, Jorge Crichigno, and Gautam Srivastava. A survey on tcp enhancements using p4-programmable devices. *Computer Networks*, 212, 05 2022.
- [HZLM16] Hai Huang, Dongyao Zhu, Kai Lu, and Chunhai Ma. DPDK-based Virtual Network Functions Acceleration in OpenStack. In *2016 12th International Conference on Mobile Ad-Hoc and Sensor Networks (MSN)*, pages 181–188, 2016.
- [OLL20] Heekuk Oh, Younghee Lee, and Bong-Jae Lee. IPsec for DPDK-enabled Virtual Network Function with High Throughput and Security. *Future Generation Computer Systems*, 104:344–351, 2020.
- [Riz12] Luigi Rizzo. netmap: a novel framework for fast packet I/O. *USENIX Annual Technical Conference (ATC '12)*, (186):1–12, 2012.
- [SAPK23] Matheus Stolet, Liam Arzola, Simon Peter, and Antoine Kaufmann. Virtuoso: High Resource Utilization and μ s-scale Performance Isolation in a Shared Virtual Machine TCP Network Stack. *arXiv preprint arXiv:2309.14016*, 2023.
- [SLW⁺20] Yu Sun, Kexin Li, Haoyuan Wang, Junfang Su, and Jianshe Wang. NetDP: A DPDK based High-Performance Network Traffic Processor. *IEEE Access*, 8:109407–109418, 2020.
- [SN16] Rinku Shah and Priyanka Naik. Kernel-bypass techniques for high-speed network packet processing CS-744. Presentation Slides, 2016.
- [SRLBA18] Gustavo Sutter, Mario Ruiz, Sergio Lopez-Buedo, and Gustavo Alonso. Fpga-based tcp/ip checksum offloading engine for 100 gbps networks. In *2018 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pages 1–6, 2018.
- [VT23] M. Vorbrodt and I. Toivanen. *Analyzing the Performance of Linux Networking Approaches for Packet Processing : A Comparative Analysis of DPDK, io_uring and the standard Linux network stack*. Dissertation, Linköping University, 2023.
- [Z⁺18] Jialong Zhou et al. XDP/DPDK-based High-speed Forwarding with Efficient Connection Management. In *2018 IEEE 26th International Conference on Network Protocols (ICNP)*, pages 1–10, 2018.